

Actividad Evaluativa - Desarrollo Backend

DISEÑO E IMPLEMENTACIÓN DE UNA API RESTFUL CON DJANGO REST FRAMEWORK



Docente: Enrique Julio Muñoz Rojas

Integrantes:

- Alexander Torres Aguirre
- Fernanda Vera González
- José Tomás Vega Ojeda
- Felipe Pérez Hidalgo
- Ignacio duran Alveal
- Brian Henry Mota Barrios
- Alexis Allendes Barraza
- Diego Lorca Bastias

Introducción

El presente informe describe las mejoras de seguridad implementadas sobre el proyecto desarrollado en la evaluación anterior. El sistema corresponde a una API REST para una clínica veterinaria desarrollada con Django REST Framework, orientada a gestionar información de pacientes, propietarios, citas médicas y registros clínicos. Debido a la sensibilidad de los datos almacenados, fue necesario incorporar mecanismos de protección que permitieran fortalecer la seguridad del sistema y reducir posibles vulnerabilidades.

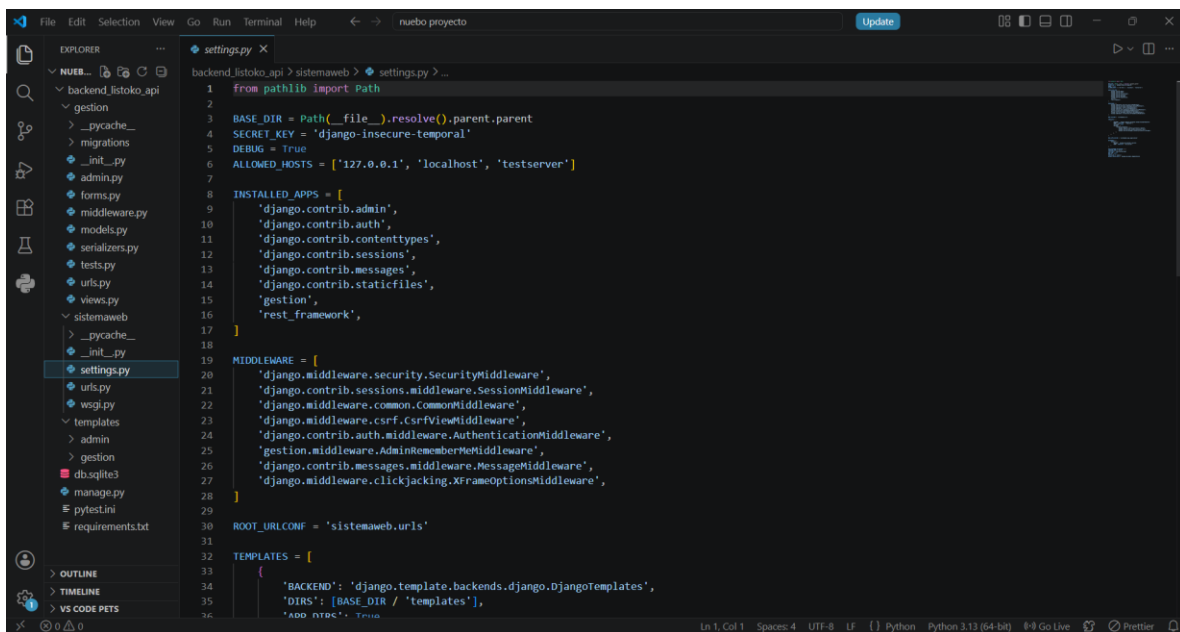
El objetivo principal de esta etapa fue mejorar la protección de la información mediante la aplicación de buenas prácticas de seguridad tanto a nivel de infraestructura como de desarrollo. Entre las principales mejoras implementadas se encuentra la incorporación de HTTPS, permitiendo que la comunicación entre clientes y servidor se realice de forma cifrada. Asimismo, se configuró una política segura de CORS, restringiendo el acceso únicamente a dominios autorizados.

También se reforzó el sistema de autenticación y autorización mediante el uso de tokens y control de permisos, asegurando que solo usuarios autorizados puedan acceder a determinadas funciones del sistema. Además, se implementaron validaciones de datos para verificar la integridad y el formato correcto de la información ingresada, reduciendo riesgos asociados a ataques comunes como la inyección de código.

Por otra parte, se incorporaron registros de eventos y herramientas de análisis de vulnerabilidades para identificar posibles debilidades y monitorear actividades relevantes dentro de la aplicación. En conjunto, estas medidas permitieron aumentar la seguridad de la API REST, logrando una aplicación más robusta, confiable y preparada para operar en entornos reales.

Análisis de riesgos

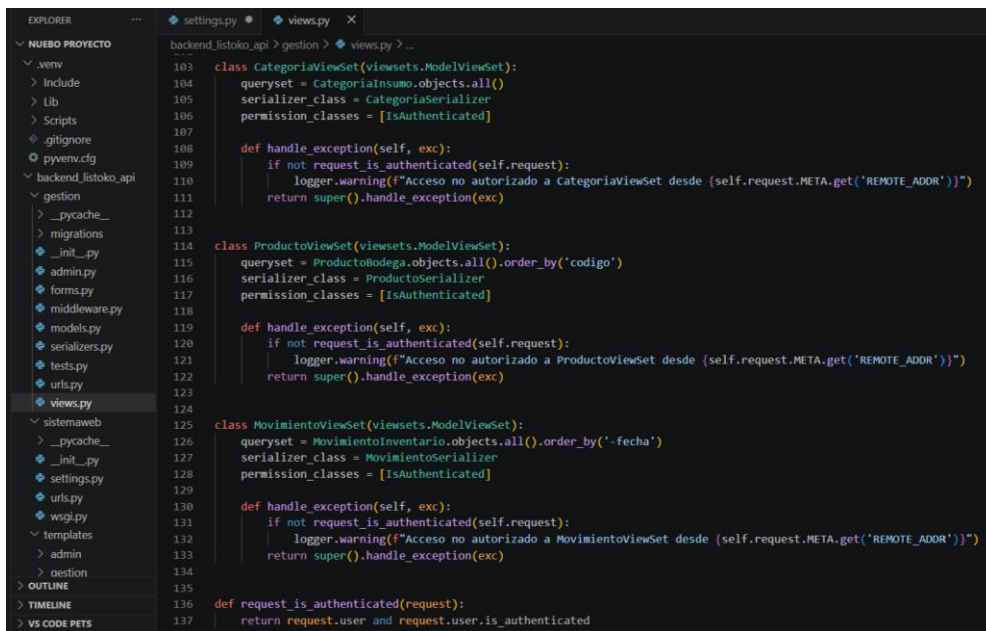
Se identificaron vulnerabilidades relacionadas con HTTPS, CORS, SQL Injection, CSRF, XSS y la ausencia de monitoreo. Un ataque exitoso podría provocar el robo de datos médicos veterinarios, modificación de citas, alteración del inventario y pérdida de confianza de los clientes. También se analizó cómo el uso del ORM de Django reduce el riesgo de SQL Injection al evitar consultas SQL construidas manualmente.



```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 SECRET_KEY = 'django-insecure-temporal'
5 DEBUG = True
6 ALLOWED_HOSTS = ['127.0.0.1', 'localhost', 'testserver']
7
8 INSTALLED_APPS = [
9     'django.contrib.admin',
10    'django.contrib.auth',
11    'django.contrib.contenttypes',
12    'django.contrib.sessions',
13    'django.contrib.messages',
14    'django.contrib.staticfiles',
15    'gestion',
16    'rest_framework',
17 ]
18
19 MIDDLEWARE = [
20     'django.middleware.security.SecurityMiddleware',
21     'django.contrib.sessions.middleware.SessionMiddleware',
22     'django.middleware.common.CommonMiddleware',
23     'django.middleware.csrf.CsrfViewMiddleware',
24     'django.contrib.auth.middleware.AuthenticationMiddleware',
25     'gestion.middleware.AdminRememberMeMiddleware',
26     'django.contrib.messages.middleware.MessageMiddleware',
27     'django.middleware.clickjacking.XFrameOptionsMiddleware',
28 ]
29
30 ROOT_URLCONF = 'sistemaweb.urls'
31
32 TEMPLATES = [
33     {
34         'BACKEND': 'django.template.backends.django.DjangoTemplates',
35         'DIRS': [BASE_DIR / 'templates'],
36         'APP_DIRS': True,
```

Propuesta técnica

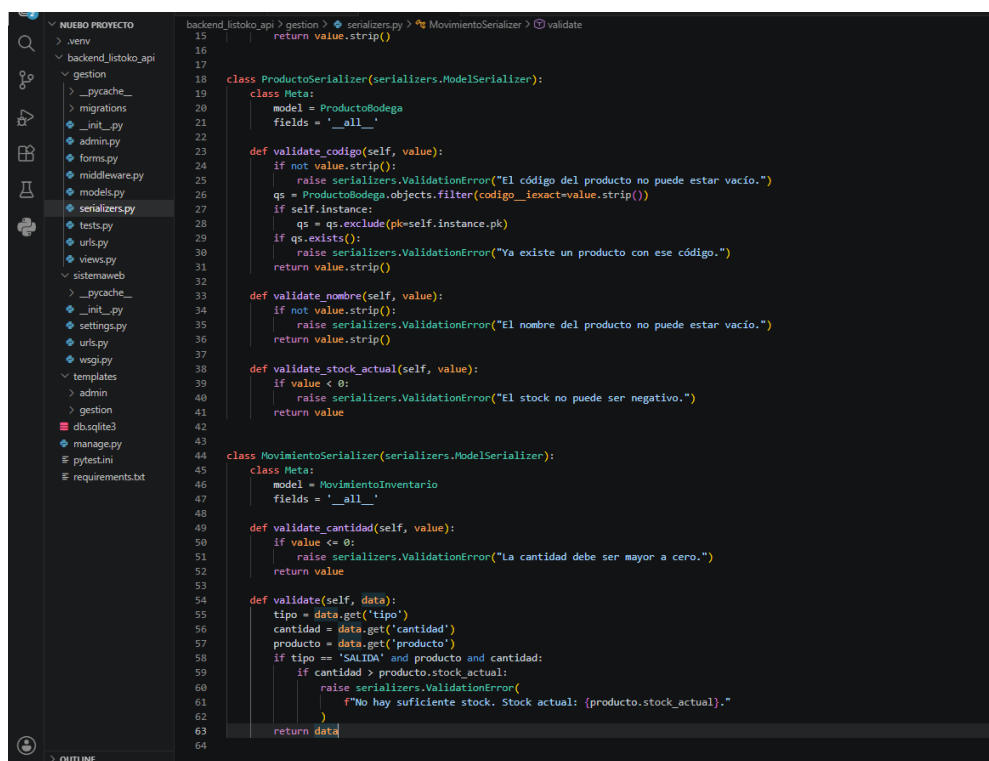
Se modificó settings.py incorporando políticas de seguridad (HTTPS, cookies seguras, CORS restrictivo y logging). En views.py se protegieron los endpoints mediante IsAuthenticated y se registran accesos no autorizados. En serializers.py se añadieron validaciones para impedir datos inválidos. En models.py se agregaron restricciones adicionales como validadores y códigos únicos para reforzar la integridad de la información.



```

103 class CategoriaViewSet(viewsets.ModelViewSet):
104     queryset = CategoriaInsumo.objects.all()
105     serializer_class = CategoriaSerializer
106     permission_classes = [IsAuthenticated]
107
108     def handle_exception(self, exc):
109         if not request_is_authenticated(self.request):
110             logger.warning(f"Acceso no autorizado a CategoriaViewSet desde {self.request.META.get('REMOTE_ADDR')}")
111             return super().handle_exception(exc)
112
113 class ProductoViewSet(viewsets.ModelViewSet):
114     queryset = ProductoBodega.objects.all().order_by('codigo')
115     serializer_class = ProductoSerializer
116     permission_classes = [IsAuthenticated]
117
118     def handle_exception(self, exc):
119         if not request_is_authenticated(self.request):
120             logger.warning(f"Acceso no autorizado a ProductoViewSet desde {self.request.META.get('REMOTE_ADDR')}")
121             return super().handle_exception(exc)
122
123 class MovimientoViewSet(viewsets.ModelViewSet):
124     queryset = MovimientoInventario.objects.all().order_by('-fecha')
125     serializer_class = MovimientoSerializer
126     permission_classes = [IsAuthenticated]
127
128     def handle_exception(self, exc):
129         if not request_is_authenticated(self.request):
130             logger.warning(f"Acceso no autorizado a MovimientoViewSet desde {self.request.META.get('REMOTE_ADDR')}")
131             return super().handle_exception(exc)
132
133 def request_is_authenticated(request):
134     return request.user and request.user.is_authenticated

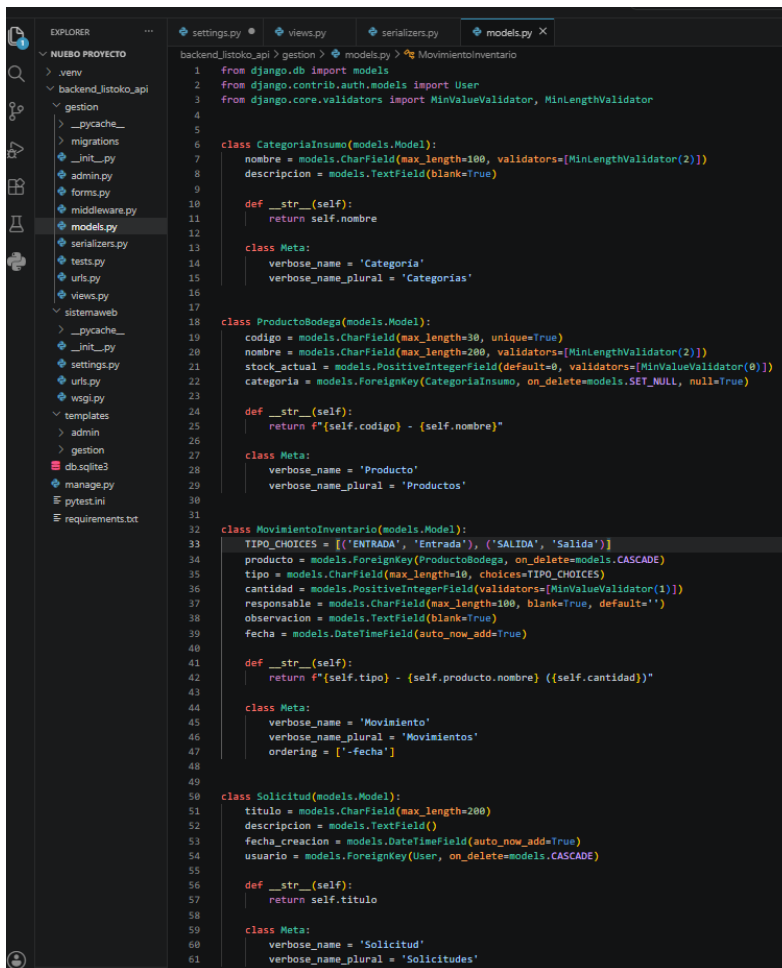
```



```

15 return value.strip()
16
17 class ProductoSerializer(serializers.ModelSerializer):
18     class Meta:
19         model = ProductoBodega
20         fields = '__all__'
21
22     def validate_codigo(self, value):
23         if not value.strip():
24             raise serializers.ValidationError("El código del producto no puede estar vacío.")
25         qs = ProductoBodega.objects.filter(codigo__iexact=value.strip())
26         if self.instance:
27             qs = qs.exclude(pk=self.instance.pk)
28         if qs.exists():
29             raise serializers.ValidationError("Ya existe un producto con ese código.")
30         return value.strip()
31
32     def validate_nombre(self, value):
33         if not value.strip():
34             raise serializers.ValidationError("El nombre del producto no puede estar vacío.")
35         return value.strip()
36
37     def validate_stock_actual(self, value):
38         if value < 0:
39             raise serializers.ValidationError("El stock no puede ser negativo.")
40         return value
41
42 class MovimientoSerializer(serializers.ModelSerializer):
43     class Meta:
44         model = MovimientoInventario
45         fields = '__all__'
46
47     def validate_cantidad(self, value):
48         if value <= 0:
49             raise serializers.ValidationError("La cantidad debe ser mayor a cero.")
50         return value
51
52     def validate(self, data):
53         tipo = data.get('tipo')
54         cantidad = data.get('cantidad')
55         producto = data.get('producto')
56         if tipo == 'SALIDA' and producto and cantidad:
57             if cantidad > producto.stock_actual:
58                 raise serializers.ValidationError(
59                     f"No hay suficiente stock. Stock actual: {producto.stock_actual}."
60                 )
61         return data

```



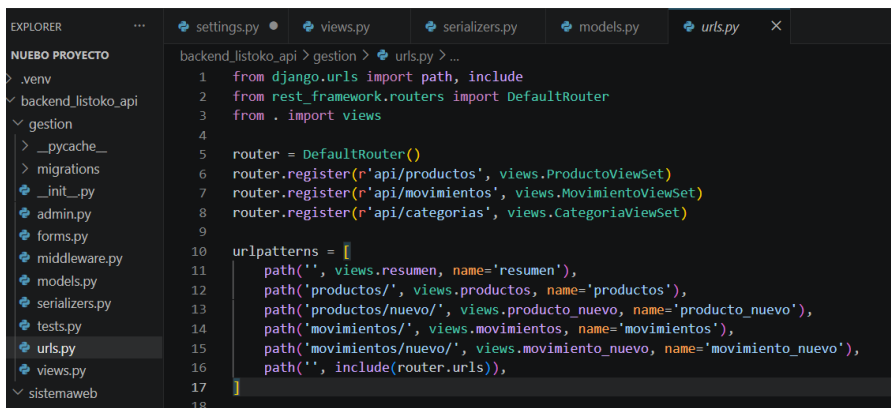
```
1 from django.db import models
2 from django.contrib.auth.models import User
3 from django.core.validators import MinValueValidator, MinLengthValidator
4
5
6 class CategoriaInsumo(models.Model):
7     nombre = models.CharField(max_length=100, validators=[MinLengthValidator(2)])
8     descripcion = models.TextField(blank=True)
9
10     def __str__(self):
11         return self.nombre
12
13     class Meta:
14         verbose_name = 'Categoria'
15         verbose_name_plural = 'Categorias'
16
17
18 class ProductoBodega(models.Model):
19     codigo = models.CharField(max_length=30, unique=True)
20     nombre = models.CharField(max_length=200, validators=[MinLengthValidator(2)])
21     stock_actual = models.PositiveIntegerField(default=0, validators=[MinValueValidator(0)])
22     categoria = models.ForeignKey(CategoriaInsumo, on_delete=models.SET_NULL, null=True)
23
24     def __str__(self):
25         return f'{self.codigo} - {self.nombre}'
26
27     class Meta:
28         verbose_name = 'Producto'
29         verbose_name_plural = 'Productos'
30
31
32 class MovimientoInventario(models.Model):
33     TIPO_CHOICES = (('ENTRADA', 'Entrada'), ('SALIDA', 'Salida'))
34     producto = models.ForeignKey(ProductoBodega, on_delete=models.CASCADE)
35     tipo = models.CharField(max_length=10, choices=TIPO_CHOICES)
36     cantidad = models.PositiveIntegerField(validators=[MinValueValidator(1)])
37     responsable = models.CharField(max_length=100, blank=True, default='')
38     observacion = models.TextField(blank=True)
39     fecha = models.DateTimeField(auto_now_add=True)
40
41     def __str__(self):
42         return f'{self.tipo} - {self.producto.nombre} ({self.cantidad})'
43
44     class Meta:
45         verbose_name = 'Movimiento'
46         verbose_name_plural = 'Movimientos'
47         ordering = ['-fecha']
48
49
50 class Solicitud(models.Model):
51     titulo = models.CharField(max_length=200)
52     descripcion = models.TextField()
53     fecha_creacion = models.DateTimeField(auto_now_add=True)
54     usuario = models.ForeignKey(User, on_delete=models.CASCADE)
55
56     def __str__(self):
57         return self.titulo
58
59     class Meta:
60         verbose_name = 'Solicitud'
61         verbose_name_plural = 'Solicitudes'
```

Las modificaciones realizadas en los distintos componentes del proyecto permitieron fortalecer significativamente la seguridad y confiabilidad de la aplicación. La configuración segura en settings.py, la protección de endpoints en views.py, las validaciones implementadas en serializers.py y las restricciones añadidas en models.py contribuyen en conjunto a reducir riesgos de acceso no autorizado, errores de integridad y vulnerabilidades comunes en aplicaciones web.

Estas mejoras permiten que la API opere de manera más segura, manteniendo una mejor protección de los datos y garantizando un funcionamiento más estable y confiable para el sistema.

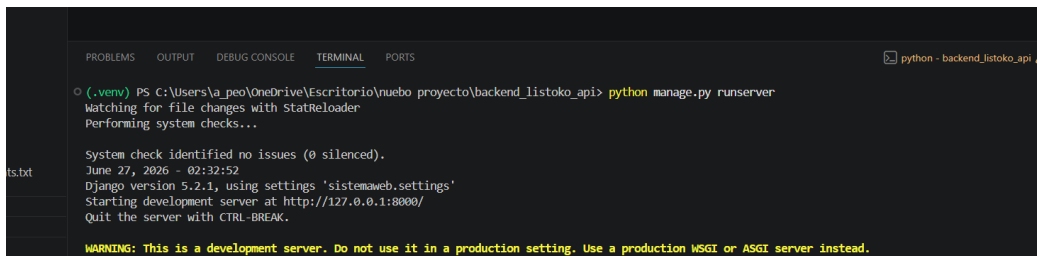
Implementación

Se reutilizó el proyecto desarrollado anteriormente por el grupo. Se actualizaron los archivos `settings.py`, `views.py`, `serializers.py`, `models.py`, `gestion/urls.py` y `sistemaweb/urls.py`. La API fue probada correctamente mediante el endpoint `/api/productos/`, obteniendo respuesta HTTP 200 OK. Además, se preparó el proyecto para ejecutar Safety y Bandit.



```
EXPLORER
  NUEVO PROYECTO
    .venv
    backend_listoko_api
      gestion
        urls.py
        ...
      ...
      migrations
      _init_.py
      admin.py
      forms.py
      middleware.py
      models.py
      serializers.py
      tests.py
      urls.py
      views.py
    sistemaweb

backend_listoko_api > gestion > urls.py > ...
1 from django.urls import path, include
2 from rest_framework.routers import DefaultRouter
3 from . import views
4
5 router = DefaultRouter()
6 router.register(r'api/productos', views.ProductoViewSet)
7 router.register(r'api/movimientos', views.MovimientoViewSet)
8 router.register(r'api/categorias', views.CategoriaViewSet)
9
10 urlpatterns = [
11     path('', views.resumen, name='resumen'),
12     path('productos/', views.productos, name='productos'),
13     path('productos/nuevo/', views.producto_nuevo, name='producto_nuevo'),
14     path('movimientos/', views.movimientos, name='movimientos'),
15     path('movimientos/nuevo/', views.movimiento_nuevo, name='movimiento_nuevo'),
16     path('', include(router.urls)),
17 ]
18
```

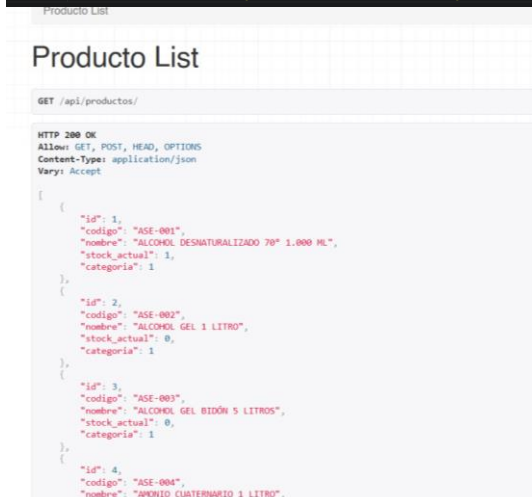


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python - backend_listoko_api

(.venv) PS C:\Users\apeo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
June 27, 2026 - 02:22:52
Django version 5.2.1, using settings 'sistemaweb.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
```



Producto List

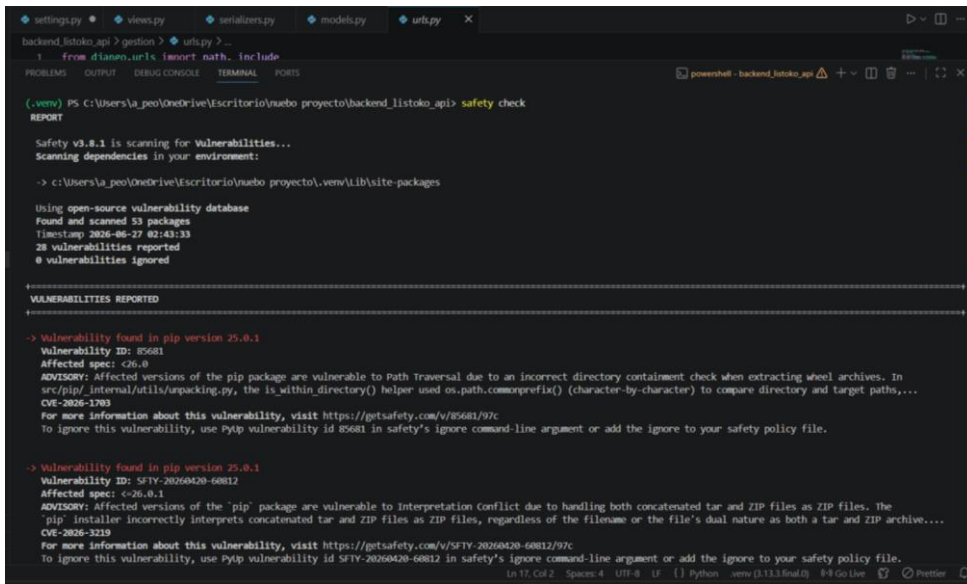
GET /api/productos/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "id": 1,
  "codigo": "ASE-001",
  "nombre": "ALCOHOL DESNATURALIZADO 70° 1.000 ML",
  "stock_actual": 1,
  "categoria": 1
},
{
  "id": 2,
  "codigo": "ASE-002",
  "nombre": "ALCOHOL GEL 1 LITRO",
  "stock_actual": 0,
  "categoria": 1
},
{
  "id": 3,
  "codigo": "ASE-003",
  "nombre": "ALCOHOL GEL BIDIÓ 5 LITROS",
  "stock_actual": 0,
  "categoria": 1
},
{
  "id": 4,
  "codigo": "ASE-004",
  "nombre": "APORRIDO CUATERNARIO 1 LITRO",
  "stock_actual": 0,
  "categoria": 1
}
```

Herramientas de análisis y monitoreo

Se propone utilizar Safety para revisar dependencias vulnerables, Bandit para analizar el código fuente y Sentry como plataforma de monitoreo de errores en producción. Estas herramientas permiten detectar vulnerabilidades de manera temprana y mejorar la seguridad del sistema.



```
settings.py  views.py  serializers.py  models.py  urls.py  X
backend_listoko_api > gestion > urls.py > ...
1 from django.urls import path, include

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - backend_listoko_api + - - - - -

(.venv) PS C:\Users\aleo\OneDrive\Escritorio\Nuevo proyecto\backend_listoko_api> safety check
REPORT

Safety v3.8.1 is scanning for Vulnerabilities...
Scanning dependencies in your environment:

-> c:\Users\aleo\OneDrive\Escritorio\Nuevo proyecto\.venv\Lib\site-packages

Using open-source vulnerability database
Found and scanned 53 packages
Timestamp: 2026-06-27 02:43:33
28 vulnerabilities reported
0 vulnerabilities ignored

VULNERABILITIES REPORTED

-> Vulnerability found in pip version 25.0.1
Vulnerability ID: RS681
Affected spec: <26.0
ADVISORY: Affected versions of the pip package are vulnerable to Path Traversal due to an incorrect directory containment check when extracting wheel archives. In src/pip/_internal/utils/unpacking.py, the is_within_directory() helper used os.path.commonprefix() (character-by-character) to compare directory and target paths,...
CVE-2026-1783
For more information about this vulnerability, visit https://getsafety.com/v/RS681/97c
To ignore this vulnerability, use Pydp vulnerability id RS681 in safety's ignore command-line argument or add the ignore to your safety policy file.

-> Vulnerability found in pip version 25.0.1
Vulnerability ID: SFTY-20260420-68812
Affected spec: <26.0.1
ADVISORY: Affected versions of the 'pip' package are vulnerable to Interpretation Conflict due to handling both concatenated tar and ZIP files as ZIP files. The 'pip' installer incorrectly interprets concatenated tar and ZIP files as ZIP files, regardless of the filename or the file's dual nature as both a tar and ZIP archive....
CVE-2026-3219
For more information about this vulnerability, visit https://getsafety.com/v/SFTY-20260420-68812/97c
To ignore this vulnerability, use Pydp vulnerability id SFTY-20260420-68812 in safety's ignore command-line argument or add the ignore to your safety policy file.
```

```

1 from django.urls import path, include

(.venv) PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api> bandit -r .
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.13.3
Run started:2026-06-27 06:48:06.601979+00:00

Test results:
>> Issue: [B105:hardcoded password string] Possible hardcoded password: 'django-insecure-temporal'
Severity: Low Confidence: Medium
CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
Location: .\sistemaweb\settings.py:4:13
3 BASE_DIR = Path(_file_.resolve()).parent.parent
4 SECRET_KEY = 'django-insecure-temporal'
5 DEBUG = True

-----

Code scanned:
Total lines of code: 509
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
  Undefined: 0
  Low: 1
  Medium: 0
  High: 0
Total issues (by confidence):
  Undefined: 0
  Low: 0
  Medium: 1
  High: 0
Files skipped (0):
(.venv) PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api>
  
```

```

PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\venv\Scripts\activate.ps1")
(.venv) PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto> cd "C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api"
>> dir seguridad.log

Directorio: C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api

Mode                LastWriteTime         Length Name
----                -
-a----             27-06-2026          2:58           56 seguridad.log

(.venv) PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api> Get-Content seguridad.log
Acceso no autorizado a ProductViewSet desde 127.0.0.1
(.venv) PS C:\Users\alejo\OneDrive\Escritorio\nuevo proyecto\backend_listoko_api>
  
```

La incorporación de herramientas como Safety, Bandit y Sentry permite complementar las medidas de seguridad implementadas en el proyecto mediante el análisis preventivo y el monitoreo continuo de la aplicación. Safety facilita la detección de dependencias vulnerables, Bandit ayuda a identificar posibles problemas de seguridad en el código fuente y Sentry permite supervisar errores y fallos en tiempo real dentro del entorno de producción. El uso conjunto de estas herramientas contribuye a mejorar la estabilidad, confiabilidad y protección general del sistema, permitiendo detectar y corregir vulnerabilidades de forma más rápida y eficiente.

Reflexión final

La seguridad debe integrarse desde las primeras etapas del desarrollo de software, ya que implementar medidas de protección de manera temprana permite prevenir vulnerabilidades y reducir riesgos futuros. Aplicar mecanismos como autenticación segura, validación de datos, monitoreo de eventos y buenas prácticas de configuración contribuye significativamente a mejorar la confiabilidad, estabilidad y protección del sistema frente a posibles amenazas o accesos no autorizados.

Las modificaciones realizadas sobre el proyecto anterior permitieron fortalecer distintos aspectos de la seguridad de la API REST desarrollada para la clínica veterinaria, incorporando controles que ayudan a proteger la información clínica y los datos de los usuarios. Además, el uso de herramientas de análisis y monitoreo permitió identificar posibles vulnerabilidades y mantener un mejor control sobre el funcionamiento de la aplicación. En conjunto, estas mejoras no solo permiten cumplir con los requisitos establecidos en la actividad, sino que también contribuyen al desarrollo de una aplicación más segura, robusta y preparada para entornos reales.



Bibliografía (APA 7)

- Django Software Foundation. (2025). Django documentation. <https://docs.djangoproject.com/>
- Django REST Framework. (2025). Django REST Framework. <https://www.django-rest-framework.org/>
- Mozilla. (2024). Web Security. <https://developer.mozilla.org/>
- OWASP Foundation. (2021). OWASP Top 10. <https://owasp.org/www-project-top-ten/>